Mugda Revisited: Sized Types, Size-Change Termination, and the Pattern-Refinement Invariant in Lean 4

Karl Mehltretter
Original Diploma Thesis: LMU Munich, December 2007 / April 2008 (Advisor: Andreas Abel)
Constructive Lean 4 Formalization & Metatheory, 2026
https://github.com/kmehltretter82/mugda

2026

Abstract

Mugda (2007/2008) was, to our knowledge, the first implemented proof assistant to combine *Martin-Löf dependent type theory* and Coquand-style *dependent pattern matching* with *Lee–Jones–Ben-Amram Size-Change Termination (SCT)* (including the nested matrix semiring ORDER_+) and *explicit* Size *indices* (`$ i` for size successor and `#` for the closure ordinal ∞) to check both *termination* of inductive definitions (`fun`) and *productivity* of corecursive definitions (`cofun`).

Eighteen years later, we revisit Mugda through the lens of transfinite ordinal semantics and machine-checked formalization in **Lean 4**. First, we conduct a complete audit of the 2007/2008 system, identifying seven soundness and completeness bugs in the interaction between dependent pattern unification, size subtyping, ORDER_+ matrix comparison, runtime size erasure, and open evaluation of overlapping clauses. Second, we isolate why naive pattern unification on size indices fails at limit ordinals ($\omega, \omega+2$): matching $x : \mathsf{D}\ \vec{u}\ \alpha$ against a constructor of target type $\mathsf{D}\ \vec{u}\ (\$i)$ justifies only the *existential ordinal inequality* $\exists \beta < \alpha,\ \$\beta \leq \alpha$, and *not* the syntactic equality $\alpha = \$\beta$. Third, we formulate and prove the **Pattern-Refinement Invariant** (`checkNoccEnv` + `checkNoccDomains` + parameter `nocc`), showing that substituting $\alpha \mapsto \$\beta$ during pattern checking is sound across all transfinite ordinals if and only if the refined size variable occurs only in the covariant return type after matching. Fourth, we upgrade pattern-clause evaluation from 2-valued `Option Env` to 3-valued `MatchResult` (`matched` | `mismatch` | `stuck`), preventing neutral variables from falling through constructor clauses into catch-all clauses during dependent typechecking. Finally, we port the entire Mugda compiler into the *pure constructive subset of Lean 4* (`Mugda/Core.lean`, with zero `partial`, zero `unsafe`, and zero `sorry`) and verify over 65 semantic and in-place algorithmic metatheorems covering every compiler pass in `Mugda/Soundness.lean` and `Mugda/Metatheory.lean`.

1 Introduction

Single-argument syntactic guard conditions reject many natural recursive and corecursive programs:

1. functions that *permute arguments* (`add2 x (succ y) = succ (add2 y x)`) or descend lexicographically on nested tuples (`ack (Pair (succ x) (succ y))`, `addpair`),

2. functions that recur on the result of a *size-preserving helper* (Euclidean `div` and `gcd` via `sub`, `quicksort` via `pivot`, `mergesort` via `split`),

3. functions that pass a *partially applied recursive call* to a higher-order combinator (`flat` on rose trees via `mapL`), and

4. *corecursive stream and stream-processor definitions* (`fib`, `eat`/`ieat`) where corecursive calls occur under continuations or mutually recursive helpers.

Mugda (Mehltretter, 2007/2008 [1]) unified these paradigms by combining:

- **Syntactic Size-Change Termination in ORDER_+ [2]:** Call extraction constructs call matrices over the recursive order algebra

$$o \in \mathsf{Order} ::= < \mid \leq \mid ? \mid \mathsf{Mat}(\mathsf{Matrix\ Order}),$$

  computes the idempotent closure under matrix multiplication, and checks that every idempotent call $M = M \cdot M$ has a strictly decreasing diagonal entry (`isDecr`).

- **Type-Based Termination and Productivity via Size [3, 4]:** Sized inductive families $\mathsf{D}\ \vec{u}\ \alpha$ are covariant in α ($\alpha \leq \$\alpha \leq \#$), whereas sized coinductive families $\mathsf{C}\ \vec{u}\ \alpha$ are contravariant in α ($\# \leq \$\alpha \leq \alpha$).

2 Transfinite Ordinal Semantics and the Limit-Ordinal Gap

2.1 Cumulative Ordinal Interpretation

In our Lean 4 formalization (`Mugda/Soundness.lean`), transfinite size ordinals below $\omega \cdot 2$ are represented by the inductive type $\mathsf{SizeOrd}$ with constructors $\mathsf{fin}(n)$ ($n \in \mathbb{N}$) and $\mathsf{omegaPlus}(k)$ ($\omega + k$), extended with the top closure ordinal $\# = \infty$ in $\mathsf{ExtSize}$. Successor on $\mathsf{ExtSize}$ satisfies $\$\infty = \infty$ (`succ_infty` and `sinfty_infty`).

For a monotone operator F_D and a coinductive operator F_C, the semantic interpretations at ordinal α are:

$$[\![\mathsf{D}\ \vec{u}\ \alpha]\!] = \bigcup_{\beta<\alpha} F_\mathsf{D}(\vec{u},\ [\![\mathsf{D}\ \vec{u}\ \beta]\!])\,, \qquad [\![\mathsf{C}\ \vec{u}\ \alpha]\!] = \bigcap_{\beta<\alpha} F_\mathsf{C}(\vec{u},\ [\![\mathsf{C}\ \vec{u}\ \beta]\!])\,.$$

At the closure ordinal ∞, the inductive interpretation stabilizes as a fixed point: $[\![\mathsf{D}\ \vec{u}\ (\$\infty)]\!] = [\![\mathsf{D}\ \vec{u}\ \infty]\!]$ (proved in `inOExt_S_iff`). Because Mugda enforces $\infty \not\leq \alpha$ for every generic size variable α (`leqSize VInfty (VGen k) = error`), it avoids the $\infty < \infty$ inconsistency.

2.2 Why Naive Pattern Unification Fails at Limit Ordinals

Every constructor c of a sized inductive family D has a target type of the form $\mathsf{D}\ \vec{u}\ (\$i)$. When checking a pattern $c\,i\,\vec{y}$ against an expected domain $\mathsf{D}\ \vec{u}\ \alpha$, Mugda's pattern unifier `inst` unifies α with $\$i$, yielding the substitution $\sigma = [\alpha \mapsto \$i]$.

In 2008 Mugda, σ was applied globally across the context Γ, the remaining argument telescope Δ, and the return type $R(\alpha)$. However, in the ordinal semantics, $x \in [\![\mathsf{D}\ \vec{u}\ \alpha]\!]$ guarantees only:

$$\exists \beta < \alpha, \quad x \in F_\mathsf{D}(\vec{u},\ [\![\mathsf{D}\ \vec{u}\ \beta]\!]) \quad \text{where} \quad \beta < \alpha \implies \$\beta \leq \alpha.$$

When α is a limit ordinal (e.g. $\omega = \mathsf{omegaPlus}(0)$, proved to have no predecessor in `omega_is_limit`), $\alpha \neq \$\beta$ for all β. This gap between the true inequality $\$\beta \leq \alpha$ and the syntactic substitution $\alpha := \$\beta$ causes two distinct soundness failures:

1. **Deep Sibling Synchronization at $\alpha = \omega + 2$ (Bug 1a, `cody_simplified.ma` [5]):** For the Brouwer ordinal type $\mathsf{O} : \mathsf{Size} \to \mathsf{Set}$ with constructors $\mathsf{Z}, \mathsf{S}, \mathsf{L}, \mathsf{M}$, consider at $\alpha = \omega + 2$:

$$t = \mathsf{M}\ (\omega+1) \quad \underbrace{(\mathsf{L}\ \omega\ (\lambda n.\, \mathsf{S}^n\ \mathsf{Z}))}_{\in [\![\mathsf{O}\ (\omega+1)]\!],\ \text{witness}\ \beta_1=\omega} \quad \underbrace{(\mathsf{S}\ 1\ (\mathsf{S}\ 0\ \mathsf{Z}))}_{\in [\![\mathsf{O}\ 2]\!] \subseteq [\![\mathsf{O}\ (\omega+1)]\!],\ \text{witness}\ \beta_2=1}\,.$$

   Although $t \in [\![\mathsf{O}\ (\omega+2)]\!]$, matching both children against the deep pattern `M .($ ($ i)) (L .($ i) f) (S .($ i) (S i z))` forces $\beta_1 = \omega$ to equal $\$i$ for some i, which is false (**Theorem** `unguarded_deep_match_unsound` in `Mugda/Soundness.lean`).

2. **Retroactive Context Refinement and Parameter Smuggling (Bug 1b & Bug 2, `loop_wrapped.ma`):** If a previous hypothesis in Γ has type $f : \mathsf{SNat}\ j \to \bot$ (or is wrapped inside a strictly positive parameter of $\mathsf{Wrap}\ (+A : \mathsf{Set}) : \mathsf{Size} \to \mathsf{Set}$), matching a subsequent argument $x : \mathsf{SNat}\ j$ against $\mathsf{succ}\ i\ y$ refines $j \mapsto \$i$ inside Γ, upgrading f from $\mathsf{SNat}\ j \to \bot$ to $\mathsf{SNat}\ (\$i) \to \bot$ (**Theorems `checkNoccEnv_tightness` and `wrap_not_covariant_without_nocc`**).

3 The Pattern-Refinement Invariant

Invariant 3.1 (Pattern-Refinement Invariant)**.** *Whenever pattern unification refines a generic size variable j to a successor term $\$v$ (`isNewSuccRefinement`) while checking pattern p against domain A in context Γ with remaining telescope $\Delta \to R$:*

1. ***Context Non-Occurrence (`checkNoccEnv`):*** *j does not occur free in any type in Γ.*
2. ***Remaining Domain Non-Occurrence (`checkNoccDomains`):*** *j does not occur free in any domain of Δ (including remaining constructor argument domains).*
3. ***Parameter Non-Occurrence (`szUsed' / szSizeVarTarget`):*** *In every sized datatype constructor and admissible function signature, the size variable does not occur in any datatype parameter `rargs`.*

Theorem 3.2 (Soundness and Tightness of the Pattern-Refinement Invariant)**.** *Under the Pattern-Refinement Invariant, after matching p, the refined size variable j occurs only in the return type $R(j)$, which by function admissibility (`szCheckIndFun`) is covariant in j. For every ordinal α and every witness $\beta < \alpha$, since $\$\beta \le \alpha$ and R is monotone:*

$$[\![R(\$\beta)]\!] \subseteq [\![R(\alpha)]\!].$$

*Hence proving $R(\$\beta)$ for the constructor's subterms soundly establishes $R(\alpha)$ across all successor and limit ordinals (**Theorems `fixed_pattern_refinement_sound`, `multi_arg_pattern_refinement_sound`, and `sized_recursion_terminates`**). Conversely, dropping any of (1), (2), or (3) admits a formal counterexample (**Theorems `checkNoccEnv_tightness`, `unguarded_deep_match_unsound`, and `wrap_not_covariant_without_nocc`**).*

4 Algorithmic Verification in Constructive Lean 4

4.1 3-Valued Clause Reduction on Open Terms (`MatchResult`)

In the 2008 Haskell prototype (`Value.hs`), pattern matching `match : Pat -> Val -> Maybe Env` returned a 2-valued `Maybe Env`, conflating:

1. a **definite constructor mismatch** (`zero` vs. `succ y`, where `matchClauses` should advance to the next clause), and
2. an **indeterminate match on a neutral variable** (`zero` vs. $\mathsf{VGen}\ k$, where reduction must block).

On overlapping clauses such as `isz zero = tt; isz x = ff`, evaluating `isz n` on an open variable $n : \mathsf{Nat}$ fell through the first clause and reduced to `ff`, allowing `bad (n : Nat) : Eq Bool (isz n) ff = refl Bool ff` and `bad zero : Eq Bool tt ff` (`overlap_bug.ma`).

In `Mugda/Core.lean`, we replace `Option Env` in `matchPat`, `matchList`, and `matchClause` with the 3-valued inductive type:

$$\mathsf{MatchResult}\ \alpha\ ::=\ \mathsf{matched}(v : \alpha)\ \mid\ \mathsf{mismatch}\ \mid\ \mathsf{stuck}.$$

When a constructor pattern ConP or successor pattern SuccP inspects a distinct constructor ($c \neq c'$), it returns mismatch and `matchClauses` advances; when it inspects a neutral variable VGen k, it returns stuck and `matchClauses` halts immediately (**Theorems** `matchClauses_stuck_blocks_fallthrough`, `matchClauses_mismatch_advances`, `matchClauses_conP_vgen_never_falls_through`, **and** `exhaustive_claus`

4.2 Comprehensive Pass Verification

The entire Mugda compiler is implemented in the pure constructive subset of Lean 4 in `Mugda/Core.lean` (zero `partial`, zero `unsafe`, zero `sorry`). Table 1 summarizes the machine-checked metatheorems in `Mugda/Soundness.lean` and `Mugda/Metatheory.lean`.

Module	Lean 4 Theorem	Verified Property
Soundness	`unguarded_deep_match_unsound`	Limit-ordinal refutation of Bug 1a at $\omega + 2$
Soundness	`fixed_pattern_refinement_sound`	Single-arg Pattern-Refinement soundness
Soundness	`multi_arg_pattern_refinement_sound`	Multi-arg telescope refinement soundness
Soundness	`sized_recursion_terminates`	Transfinite well-founded sized recursion
Soundness	`inOExt_S_iff`	Initial algebra stabilization at ∞ (`#`)
Soundness	`wrap_covariant_of_nocc`	Covariance of `Wrap` under parameter `nocc`
Soundness	`wrap_not_covariant_without_nocc`	Tightness of Bug 2 parameter `nocc`
Soundness	`checkNoccEnv_tightness`	Tightness of `checkNoccEnv`
Soundness	`inStream_contravariant`, `fibFrom_productive`	Coinductive stream contravariance & productivity
Metatheory	`leqSize_iterVSucc_gen/both`	Universal $\$^n$ tower subtyping laws
Metatheory	`inst_iterVSucc_infty`	Universal $\$^n$ tower stripping against `#`
Metatheory	`matchPat_eraseSizes_comm`	Universal size-erasure commutation for `matchPat`
Metatheory	`succP_breaks_size_erasure`	Tightness of Bug 5 (`cp (.SuccP _) = false`)
Metatheory	`matchClauses_conP_vgen_never_falls_through`	3-valued `MatchResult` neutral stuckness barrier
Metatheory	`exhaustive_clauses_nullary_progress`	Progress theorem for constructor-covering clauses
Metatheory	`whnf_atomic_fuel_indep`, `whnf_deterministic`	Fuel invariance & determinism of NbE evaluation
Metatheory	`nocc_substVal_id_of_normSizeOrAtom`	Universal `nocc` $\Longrightarrow$ `substVal` identity
Metatheory	`checkNoccEnv_forall_mem`, `checkNoccDomains_pi`	Universal soundness of `checkNocc*`
Metatheory	`spos_vpi_inv`, `sposConstructor_vpi_inv`	Strict positivity inversion on constructor telescopes
Metatheory	`eqVal'_iterVSucc_vgen_refl`, `leqApp_sized_variance`	Tower reflexivity & variance of `eqVal'`/`leqVal'`
Metatheory	`szSizeVarTarget_enforces_rargs_nocc`	Bug 2 parameter `nocc` enforcement in `cofun` targets
Metatheory	`checkPattern_conP_inv`, `checkDot_inv`	Full inversion of `checkPattern (.ConP)` & `checkDot`
Metatheory	`syntactic_checkPattern_conP_sound_in_ordinal_model`	Bridge: `Core.checkPattern` → `Soundness` ordinal mod
Metatheory	`checkType_pi_inv`, `checkExpr_llet_inv`	Bidirectional typing inversion for Π and `let`
Metatheory	`compScalar_maxScalar_distrib_left/right`	Scalar Order Kleene-semiring laws
Metatheory	`maxO_mat_never_mat`, `collapseOrder_never_mat`	Bug 3b matrix collapse invariants
Metatheory	`isDecr_mat_iff`, `checkAll_sound`	Universal SCT diagonal witness characterization
Metatheory	`compScalar_sound_sizeOrd`, `no_infinite_decr_sizeOrd`	Ordinal soundness & well-foundedness of Order
Metatheory	`collectCalls_llet_includes_bound`	Bug 4 `LLet` call-extraction completeness

Table 1: Summary of machine-checked theorems in `Mugda/Soundness.lean` and `Mugda/Metatheory.lean`.

5 Case Studies: `SizedLib` Verified Compiler and Coinductive Bisimulation

To evaluate Mugda on larger developments beyond the 2007 thesis benchmarks, we developed a 9-chapter progressive tutorial (`tutorial/`, backed by 38 verified `.ma` files) and two larger verification case studies:

1. `SizedLib` + **Verified Expression Compiler (`examples/showcase/sizedlib/sizedlib.ma`, ∼1,100 lines):** Combines sized arithmetic (`sub`, `div`, `mod`, `gcd`), sized list sorting (`mergeSort`, `quickSort`), rose trees (`Rose`, `mapRose`, `foldRose`), coinductive streams (`fibs`, `hamming`), stream processors (`SP`), and a machine-checked stack-machine compiler and constant-folding optimizer for arithmetic expressions (`compile_correct`, `compileOpt_correct`).

2. **Coinductive Stream Bisimulation (`examples/showcase/bisim/bisim.ma`):** Defines sized coinductive bisimulation $\mathsf{Bisim}\ A\ i\ s_1\ s_2$ ($s_1 \approx_i s_2$), proves by productive corecursion (`cofun`) that $\approx_\#$ is an equivalence relation (`bisimRefl`, `bisimSymm`, `bisimTrans`), verifies stream functor and map-fusion laws ($\mathsf{map}\ g\ (\mathsf{map}\ f\ s) \approx \mathsf{map}\ (g \circ f)\ s$ and $\mathsf{interleave}\ (\mathsf{evens}\ s)\ (\mathsf{odds}\ s) \approx s$), and proves by induction on $n : \mathsf{Nat}$ that bisimilar streams have propositionally equal n-th elements (`bisimNthEq`) and finite prefixes (`bisimTakeEq`).

References

[1] Karl Mehltretter. *Termination Checking for a Dependently Typed Language.* Diploma thesis, LMU Munich, December 2007 (supervised by Andreas Abel).

[2] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The Size-Change Principle for Program Termination. In *POPL 2001*, pp. 81–92. ACM, 2001.

[3] Andreas Abel. *A Polymorphic Lambda-Calculus with Sized Higher-Order Types.* PhD thesis, LMU Munich, 2006.

[4] Andreas Abel. MiniAgda: Integrating Sized and Dependent Types. In *PAR 2010*, EPTCS 43, pp. 14–28, 2010.

[5] Cody Roux. Sized Types and Deep Pattern Matching at Limit Ordinals. Agda Issue Tracker (#1946 and #2820), 2016/2021.